Final report on the project "Solving the Subset Sum Problem via Quantum Walk Search"

A study and reproduction attempt of the MNRS quantum walk implementation for the Constrained Subset Sum Problem.

https://github.com/jacopobellosi/Solving-the-Subset-Sum-Problem-via-Quantum-Walk-Search

Jacopo Bellosi — Software Engineering 2, Politecnico di Milano

April 2026

Contents

1 Introduction

1.1 Goal of the project

The objective of this project was to reproduce the results of the paper *"Solving the Subset Sum Problem via Quantum Walk Search"* (Lancellotti, Perriello, Barenghi, Pelosi; IEEE Transactions on Computers, vol. 75, n. 1, 2026). The paper proposes a gate-level quantum algorithm for the

Constrained Subset Sum Problem (CSSP) and provides a reference implementation in `myQLM`; the assignment was to read the paper and the code thoroughly, run the algorithm on an instance small enough to be simulated classically, and verify whether the simulated output matched the expected behaviour. When the output turned out to be flat over all admissible solutions, the project shifted to a structured diagnosis of the cause and to the production of the smallest correctness fixes that could be agreed with the paper authors.

1.2 The paper at a glance

The Subset Sum Problem (SSP) asks whether a subset of a given multiset of integers sums to a target value p. The *constrained* variant (CSSP) further requires the subset to contain exactly k elements. SSP is NP-hard and underlies several post-quantum cryptographic constructions, so it is a natural target for quantum algorithms. The paper attacks CSSP with a Magniez–Nayak–Roland–Santha (MNRS) quantum walk on the Johnson graph $J(n, k)$, whose vertices are the candidate k-subsets and whose edges connect subsets that differ in one element. The walk is interleaved with a sum-checking oracle and with a QPE-based approximate reflection so that, after a number of outer iterations of order $\sqrt{N/M}$ with $N = \binom{n}{k}$ and M the number of solutions, the amplitude on the marked vertex grows.

1.3 Structure of the report

- Section 2 summarises the paper and the observed behaviour of the unmodified reference code on the smallest instance.
- Section 3 documents the five modifications applied to the code, in chronological order, with the corresponding diffs and simulation results.
- Section 4 compares seven related works and explains, in plain terms, why their algorithms can be simulated end-to-end while the one in the paper cannot.
- Section 5 presents the structured mapping between the paper and the implementation, summarising the issues that were addressed and those that remain open.
- Section 6 draws the conclusions.

2 The paper and the original behaviour

2.1 Problem statement

Given a multiset $X = \{x_1, \ldots, x_n\}$ of positive integers and a target $p \in \mathbb{Z}$, the Subset Sum Problem asks whether there exists $S \subseteq X$ with $\sum_{x \in S} x = p$. The *constrained* variant studied in the paper additionally requires $|S| = k$. The candidate subsets are then exactly the vertices of the Johnson graph $J(n, k)$, whose vertex set has cardinality $N = \binom{n}{k}$ and whose edges connect pairs of vertices that differ in a single element. A brute-force classical search costs $O(N)$ oracle calls; the paper's contribution is a quantum algorithm with the standard quadratic order $O(\sqrt{N})$, but designed inside the MNRS framework so that the explicit gate count and depth are smaller than a direct Grover construction in the cryptographic regime ($n \geq 25$ for lower T-count, $n \geq 2^{13}$ for lower T-depth).

2.2 The MNRS pipeline

The paper builds the algorithm from four components.

(1) Johnson graph $J(n,k)$. The search space of the problem is represented as a graph. Each vertex is one possible k-subset of the input list; two vertices are connected by an edge if the corresponding subsets differ in exactly one element. This graph is called the Johnson graph $J(n,k)$.

The *spectral gap* δ measures how well-connected the graph is: when δ is close to 1 the graph mixes quickly, while a smaller δ means the walk needs more steps to explore it. For the Johnson graph the formula is:

$$\delta \;=\; \frac{n}{k(n-k)}, \qquad 1 \le k \le n-1.$$

(2) Marking oracle U_f. A quantum circuit that identifies solution vertices. It adds up the elements of the current subset into an ancillary register and applies a phase flip (sign change) whenever the sum equals the target value p. Non-solution vertices are left unchanged.

(3) Walk operator U_W. The quantum version of one random-walk step on $J(n,k)$. It is built from the *update operator* U_U, which creates a superposition over all neighbours of the current vertex, and consists of two reflections:

$$U_W \;=\; U_{\mathrm{ref}}(a_{v_j})\, U_{\mathrm{ref}}(a_{v_i}), \qquad U_{\mathrm{ref}}(a_v) \;=\; U_U\, U_{\mathrm{ref}}(0)\, U_U^{\dagger}.$$

One walk step requires four applications of U_U.

(4) Approximate reflection via QPE. To amplify the probability of the solution vertex, the algorithm needs to distinguish the *stationary state* $|E\rangle$ — the state the walk converges to — from all other states. Since $|E\rangle$ is the unique state with eigenphase 0 under U_W, a Quantum Phase Estimation (QPE) circuit applied to U_W can identify it: it leaves $|E\rangle$ unchanged and flips the sign of every other component.

The number of qubits in the QPE register, ℓ_s, controls the precision of this estimation. A larger ℓ_s means finer phase resolution; the required value grows as the spectral gap δ decreases: $\ell_s = \lceil \log_2(\pi/(2\sqrt{\delta})) \rceil$. The total cost of the algorithm is

$$T_S \;+\; \frac{1}{\sqrt{\varepsilon}}\left(\frac{T_U}{\sqrt{\delta}} + T_C\right), \quad \varepsilon = M/N,$$

where T_S is the cost of preparing the stationary state, T_U the cost of one walk step and T_C the cost of one oracle call. The $1/\sqrt{\delta}$ factor is where the quadratic speed-up over classical brute force comes from.

2.3 Analytical Validation and Scope

The paper provides closed-form analytical estimates of gate count, depth and qubit count of every sub-routine; a comparison against a Grover baseline showing better T-count for $n \ge 25$ and better T-depth for $n \ge 2^{13}$; and reference open-source implementations in Qiskit and `myQLM`. It does not, however, run the algorithm end-to-end on any concrete instance: the cryptographic regime targeted by the paper is far above any classical state-vector simulator. The paper itself states that "the smallest non-trivial case ($n = 4$, $k = 2$) already requires 38 qubits, exceeding the practical limits of classical simulation", with Table III of the paper tabulating resources for $n \in \{4, \ldots, 10\}$ and $k = \lfloor n/2 \rfloor$. The algorithm is designed for fault-tolerant hardware, so validation is restricted to the building blocks taken individually.

2.4 Baseline Execution on the Smallest Instance

The first experiment consisted in a direct execution of `cssp.py` on the smallest instance compatible with classical simulation, $n = 3$, $k = 1$, `values` $= [1, 2, 3]$, `target_sum` $= 3$. The unique marked solution is $S = \{3\}$, so an amplifying algorithm should produce a distribution close to $(0, 0, 1)$ on

the vertex register S_1. Instead, the simulation returned a flat distribution, identical to the initial Dicke state, after approximately eleven hours of single-core CPU time on a circuit of 28 qubits and $\sim$2 500 gates. Table 1 shows the direct run side by side with a run instrumented with intermediate checkpoints.

(a) Direct run of `cssp.py`

```
n=3, k=1, values=[1,2,3], target=3
{'nbqbits': 28, 'gate_size': 2547}

Final measurement on S_1:
  0.33333  |1>
  0.33333  |2>
  0.33333  |3>

(runtime: ~11h CPU on a single core)
```

(b) Run with intermediate checkpoints

```
Ckpt 1  Dicke + VBE:
  6 states, p=0.167, ampl=+0.408
  uniform over the 3 subsets

Ckpt 3  Post-Oracle:
  ampl=+0.289 on Sum=2
  ampl=-0.289 on Sum=3  <-- phase flip OK

Ckpt 4-5  Post-QPE / Post-Diffusion:
  signs scrambled, no concentration

Ckpt 6  End of iteration:
  flat prob on S_1
```

Table 1: Output of the unmodified reference code on the smallest instance $J(3,1)$. The marginal distribution on S_1 is identical to the initial Dicke state. The oracle correctly flips the phase of the marked sum, but the QPE-based reflection scatters the signs instead of converting them into amplitude. The instance $J(3,1)$ sits below the range $n \geq 4$ tabulated in Table III of the paper, so the qubit count of 28 comes from our simulation.

Both compilation and simulation terminate without errors, so the problem is not a low-level implementation bug. The five modifications described in Section 3 were the sequence of attempts to localise where the amplification fails to occur, the underlying reason is explained in Section 3 after all five modifications have been described.

3 Modifications applied to the code

The modifications below are reported in chronological order. Every entry contains the rationale of the fix, the relevant extract of the diff and the final measurement obtained after applying the modification on top of the previous one. All runs use the same smallest instance $J(3,1)$ with target $p = 3$, so that the gate counts and the output distributions are directly comparable. The corresponding output logs of every run are preserved in the repository.

Modification 1 — Symmetric reflections A and B on the full coin (17 March 2026)

What this fix is about. Inside one walk step, the algorithm reflects the state around $|0\ldots0\rangle$ on a coin register of n qubits. In the reference code this reflection acted only on half of the coin at a time, which is a different operation and breaks the walk operator.

Each reflection in the walk operator should apply a phase flip controlled on the *full* coin register $\omega = (\omega_1, \omega_0)$, which contains n qubits (k in `wstate_ones` and $n-k$ in `wstate_zeros`). In the original code, Reflection A controlled the phase flip only on `wstate_ones` via `Z.ctrl(k)`, and Reflection B only on `wstate_zeros`, with an additional typo on one of the surrounding X loops. The fix makes both reflections control on the full n-qubit coin.

```
# --- Before (Reflection B) ---
for j in range(n - k):
    prw.apply(X, wstate_zeros[j])
prw.apply(Z.ctrl(n - k), qpe_s[qw_iter], wstate_zeros)
for j in range(k):                              # BUG: should be range(n-k)
```

```
    prw.apply(X, wstate_zeros[j])

# --- After (17 March 2026) ---
for j in range(k):     prw.apply(X, wstate_ones[j])
for j in range(n - k): prw.apply(X, wstate_zeros[j])
prw.apply(Z.ctrl(n), qpe_s[qw_iter],
          wstate_ones, wstate_zeros)        # control on the full coin
for j in range(k):     prw.apply(X, wstate_ones[j])
for j in range(n - k): prw.apply(X, wstate_zeros[j])
```

```
n=3, k=1, values=[1,2,3], target=3
{'nbqbits': 28, 'gate_size': 2581}
0.33333  |1>     0.33333  |2>     0.33333  |3>
```

This is a genuine bug: the original code does not implement the double reflection of the paper. The output distribution, however, is still flat: the fix is necessary for correctness, but on this smallest instance it is not the cause of the missing amplification.

Modification 2 — Removal of the α/ω cleanup block (20–23 March 2026)

What this fix is about. The QPE block in the original code tried to manually reset two ancillary registers (α and ω, used during the walk steps) to zero before the final QFT. The reset routine is correct only if those registers are already in $|0\rangle$, but at that point in the circuit they are entangled with the visited vertices, so the manual reset corrupts the state instead of cleaning it. The correct uncomputation is automatic in `myQLM` and the manual block is therefore removed.

After the walk steps inside the QPE block, the original code tried to manually reset two ancillary registers (α and ω) back to $|0\rangle$. The problem is that at that point those registers are *entangled* with the current vertex: they no longer hold an independent value that can simply be erased. Re-applying `generate` to an entangled register corrupts the quantum state instead of cleaning it. The `myQLM` framework already handles this correctly via the `prw.uncompute()` call at the end of the `compute()` block, so the manual cleanup is both redundant and destructive.

```
# --- Before ---
for j in range(k):
    prw.apply(qregs_init.copy_register(m).ctrl(),
              wstate_ones[j], node_s_ones[j], alpha_ones)
for j in range(n - k):
    prw.apply(qregs_init.copy_register(m).ctrl(),
              wstate_zeros[j], node_s_zeros[j], alpha_zeros)
prw.apply(generate(k, 1),     wstate_ones)
prw.apply(generate(n - k, 1), wstate_zeros)

# --- After (20--23 March 2026) ---
# Block removed: uncomputation handled by prw.uncompute().
```

```
{'nbqbits': 28, 'gate_size': 1359}     # -47% gates
0.33333  |1>     0.33333  |2>     0.33333  |3>
```

The gate count drops by about 47%, confirming that the cleanup block was redundant. A surgical per-element reset of α and ω that does not touch the entangled state would be preferable to a global removal, but its design lies outside the scope of these patches and was therefore not pursued. The output distribution stays the same.

Modification 3 — Use of `insert_lw` instead of `insert_ld` (22 March 2026)

What this fix is about. The walk step inserts and deletes elements from a small sorted list. The library offered two versions of the routine, and the version selected by default left "garbage" bits in the workspace whenever `delete` was applied to its output. The fix swaps to the version that behaves as the inverse of `insert` on the same input.

The paper provides two versions of the update operator: a *low-width* variant (fewer ancilla qubits, sequential) and a *low-depth* variant (faster in principle, but needs more ancillas). For the walk to work correctly, each `insert` and its corresponding `delete` must be exact inverses of each other. A reversibility test showed that `delete` built on `insert_ld` leaves leftover values in the ancillas (the source code itself had a comment warning about this), so the composition `delete` ∘ `insert_ld` is not the identity. Over many walk steps these leftover values accumulate and corrupt the QPE register. The fix replaces `insert_ld` with `insert_lw` inside `delete`, which passes the reversibility test.

```
# --- After (22 March 2026) ---
from .sliding_sort_array import insert_lw, insert_ld
insert = insert_lw if low_width else insert_ld    # reversible default
# delete() in sliding_sort_array.py now calls insert_lw(...).dag()
```

```
{'nbqbits': 28, 'gate_size': 1366}
0.33333  |1>     0.33333  |2>     0.33333  |3>
```

This is a structural fix: the walk operator was not unitary on the ancillary subspace before the change. The output distribution is again unchanged on the smallest instance.

Modification 4 — Clamping the spectral gap and forcing a minimum QPE precision (27 March 2026)

What this fix is about. The spectral gap δ controls how many QPE qubits the algorithm allocates. On instances small enough to be simulated, the closed-form formula returns a mathematically valid number that is, however, outside the range in which the formula is meant to be applied; this leaves the QPE register too small to do its job. The fix is a defensive clamp that keeps the block well-defined on those instances.

For the smallest instance $(n, k) = (3, 1)$, the formula $\delta = n/(k(n-k))$ gives $\delta = 1.5$. This is a sign that the formula is being used outside the regime it was designed for: a valid spectral gap must lie between 0 and 1. Feeding this value into the QPE precision formula gives $\ell_s = 1$, meaning a single QPE qubit. One qubit can only distinguish two phase values, which is not enough to correctly identify the stationary state $|E\rangle$ among all eigenstates of U_W. As a defensive fix, δ is clamped to at most 1 and ℓ_s is forced to a minimum of three qubits.

```
# --- After (27 March 2026) ---
delta  = min(n / (k * (n - k)), 1.0)
len_s  = max(int(np.ceil(np.log2(np.pi / (2 * np.sqrt(delta))))), 3)
```

```
{'nbqbits': 28 -> 30, 'gate_size': 1366}
0.33333  |1>     0.33333  |2>     0.33333  |3>
```

The clamp does not create a spectral gap that the graph does not have: it only ensures that the QPE block is well-defined. Instances with $\delta < 1$ are unaffected, so the patch is backward-compatible.

Modification 5 — Diagnostic checkpoints (17 April 2026)

What this modification is about. Modifications 1 and 4 are genuine bugs confirmed upstream; Modification 2 is a local mitigation; Modification 3 is not a defect but a variant choice. Despite these changes, the output remains unchanged. To find out *where* in the circuit the amplification is being lost, the algorithm is instrumented so that the full quantum state can be inspected at six key points of one MNRS iteration.

To localise where the amplification is being lost, an instrumented copy `cssp_with_checkpoint.py` was written. It simulates the full state at six key points of one MNRS iteration (Dicke + VBE[1],

[1] Variable-Basis Encoding, the ripple-carry adder used to initialise the weight registers.

edge superposition, post-oracle, post-QPE, post-diffusion, end of iteration) and prints, for each state with $p > 5 \cdot 10^{-4}$, the probability and the complex amplitude, parsed into the named registers (Dicke, S_1, S_0, T, α, ω, QPE, Sum).

```
Ckpt 3 (Post-Oracle):
  Prob 0.0833 | Ampl +0.2887 | S_1:0 | Sum:2     (non-marked)
  Prob 0.0833 | Ampl -0.2887 | S_1:1 | Sum:3     (marked: phase flip OK)

Ckpt 5 (Post-Diffusion):
  Prob 0.0417 | Ampl +/-0.2041 | scattered signs
  (no concentration on Sum:3)
```

The trace shows two facts that are not visible from the final measurement alone. First, the oracle is correct: at Checkpoint 3 the marked states have amplitude -0.289 and the non-marked states have $+0.289$. Second, the failure sits inside the QPE-based reflection: between Checkpoint 3 and Checkpoint 5 the signs become evenly mixed, so the block that should rotate them into amplitude is acting as a sign-scrambler instead. This empirical fingerprint is the diagnostic of a QPE run with ℓ_s too small to resolve the eigenphase of $|E\rangle$ from the other eigenphases of U_W.

3.1 Why the output remains flat after the fixes

After Modifications 1–5 the marginal distribution on S_1 is still $(\frac{1}{3}, \frac{1}{3}, \frac{1}{3})$ (Table 2). The reason is not a single local bug, but a structural mismatch between the regime in which MNRS delivers a quadratic speed-up and the smallest instances that are small enough to be simulated. On $J(3,1) \equiv K_3$ the walker mixes in one classical step and the walk operator has only three eigenphases; the value of δ obtained from the formula $\delta = n/(k(n-k))$ falls outside the range of a true spectral gap, and the precision ℓ_s collapses to a single qubit, which is not enough to implement the approximate reflection. On $J(4,2)$ the gap is exactly $\delta = 1$, again a borderline case with no separation to exploit and a circuit that already crosses the practical qubit ceiling of `PyLinalg` (the state-vector backend of `myQLM`). Genuine MNRS instances such as $(5,2)$, $(6,2)$, $(6,3)$ have $\delta < 1$ and the algorithm should amplify the solution, but the qubit count grows like $\Theta(n \log n)$ and quickly exceeds the size of the simulator.

Mod	Date	Type	Effect on gates	Effect on distribution
1	17 Mar	Genuine bug fix	2547 → 2581	flat
2	20 Mar	Genuine bug (local mitigation)	2547 → 1359 (−47%)	flat
3	22 Mar	Not a defect upstream	1359 → 1366	flat
4	27 Mar	Defensive patch	1366 (28 → 30 qubits)	flat
5	17 Apr	Diagnostic tool	no change	flat

Table 2: The five modifications, in chronological order, with the measured effect on the smallest instance $J(3,1)$.

3.2 Outcome of the discussion with the paper authors

The five modifications above were brought to the attention of one of the paper authors. The exchange clarified the status of each item without entering into specific implementation details, and produced the following outcome.

- **Modification 1** (asymmetric reflections) and **Modification 4** (lower bound on ℓ_s) were confirmed as real issues and were therefore included in the upstream repository.
- The **rewritten** `README`, prompted by the documentation gap noticed during the mapping, was likewise agreed and merged.
- **Modification 2** (cleanup block) was confirmed problematic as a global removal, but the proper resolution requires a different construction (a per-element reset that does not touch the entangled

state); this falls outside the scope of a small correctness pull request and was therefore kept as a local mitigation only.

- **Modification 3** (low-width vs. low-depth update operator) is not a defect of the upstream code: the choice of variant is a deliberate trade-off discussed in the paper, and the equivalent behaviour can already be obtained by selecting the low-width variant via the existing flag. The change was therefore not propagated upstream.
- **Modification 5** (diagnostic checkpoints) is an instrumentation tool by design and was kept only in the local repository.

The pull request that consolidates Modifications 1 and 4 together with the rewritten `README` was merged into the upstream repository `paper-codes/2025-TC` as PR #1.

4 Comparison with related works

To contextualise the reproduction difficulties, seven related works were surveyed and stored together with the rest of the project material in the repository. The selection covers two quantum algorithms for SSP-like problems, three quantum-walk papers on graphs other than the Johnson graph, and two papers on Grover-style search. Table 3 summarises them and records, for each one, why the proposed algorithm is amenable to end-to-end simulation while the one in our paper is not.

Reference	Algorithm	Instance simulated	Validation platform	Small description
Zheng et al., 2022 (`s11432-021-3334-1`)	QPE + Amplitude Amplification for SSP	$\|S\|$=4, w=12, 10 qubits	Qiskit + IBM Q hardware	No quantum walk: the oracle marks the sum directly, so there is no spectral-gap dependence and no QPE precision constraint.
Sato et al., 2024 (`2405.07129v3`)	Two-step Grover for TSP, HOBO encoding	n=3, 4 cities, 6/12 qubits	Qiskit Aer	Compact encoding, no walk, the diffusion is the standard Grover one.
Perriello et al., 2021 (`ISD.pdf`)	Grover for ISD on Prange formulation	crypto sizes ($\sim 2^{32}$ qubits)	Atos QLM, subcomponent validation only	No end-to-end simulation: the paper publishes resource estimates, not measured probabilities.
Razzoli et al., 2024 (`entropy-26-00313`)	Discrete-time walk on cycles, no marking	$N = 4, 8$, hardware run	IBM ibm_cairo (27 qubits)	Free walk, no oracle and no reflection: nothing depends on the spectral gap.
da Silva et al., 2024 (`entropy-26-00668`)	Standard Grover on a database of 16 colours	5 qubits	PyLinalg + NoisyQProc	$N = 16$, well inside the regime where $\sqrt{N}$ iterations remain few.
Portugal & Moqadam, 2025 (`entropy-27-00454-v2`)	Continuous-time walk on K_N, $K_{n,n}$, Q_n	K_8, $K_{32,32}$, Q_{10}	Qiskit + Hiperwalk	Highly symmetric graphs whose spectrum is known in closed form; no QPE.
Wing-Bocanegra et al., 2025 (`s41598-025-87304-0`)	Single-step walk on K_{2^n} for QAOA initialisation	K_4 to K_{64}	Qiskit + IBM ibmq_manila	A single walk step removes both the iteration tuning and the spectral-gap dependence.

Table 3: Related quantum-search papers and the reason each one can be simulated end-to-end.

4.1 Resource budget: their experiments versus ours

The first observation that emerges from Table 3 is the size of the experiments that the surveyed works actually run. The papers that publish a measured output distribution use between 5 and 27 qubits: 5 for da Silva et al. on a 16-element database, 6–12 for Sato et al. and Zheng et al. on small

SSP- and TSP-like instances, 4–10 for the walks of Portugal & Moqadam and Wing-Bocanegra, and up to 27 qubits for the free walk of Razzoli et al. that runs on real NISQ (Noisy Intermediate-Scale Quantum) hardware. The only work in the table that does not publish a measured distribution is Perriello et al., whose targets reach roughly 2^{32} logical qubits and are therefore out of reach for any device or simulator currently available; that paper follows the same strategy as the one we studied and only publishes resource estimates.

The MNRS pipeline of the paper we studied sits in a different budget. Even after all five modifications, the smallest instance $J(3,1)$ already requires 30 qubits and roughly 1 400 gates. At the same time, $J(3,1)$ is below the regime in which MNRS provides any speed-up. The smallest instance that the paper itself considers non-trivial, $J(4,2)$ with bit-width $m = 2$ bits per element, is 38 qubits and is also the smallest one explicitly tabulated. To enter a regime in which the spectral gap δ is strictly below 1 and the algorithm has an actual amplification window to exploit, one needs at least $J(5,2)$, $J(6,2)$ or $J(6,3)$, which Table III of the paper tabulates as requiring several tens of qubits. The cryptographic regime targeted by the asymptotic analysis is much larger still: $n \geq 25$ for a T-count advantage over Grover and $n \geq 2^{13}$ for a T-depth advantage, with qubit counts that grow as $\Theta(n \log n)$.

4.2 Constraints on End-to-End Simulation

Two ceilings explain why the surveyed works can publish a measured distribution while ours cannot:

- **Classical simulator ceiling.** A state-vector simulator on a workstation with 64 GiB of RAM is limited to about 32 qubits. The works in Table 3 that use a classical simulator stay well below this ceiling. The MNRS implementation already exceeds it on its smallest tabulated instance (38 qubits), which is why the authors do not publish a simulated output and why we had to drop down to $J(3,1)$.
- **Real hardware ceiling.** Current NISQ devices offer ~ 100 noisy qubits with gate error rates of $\sim 10^{-3}$. The papers that run on hardware all use circuits that are either shallow (a single walk step, no reflection) or built on small registers where the error budget is manageable. The MNRS pipeline targets a fault-tolerant device with effective error rates of order 10^{-15} and gate depths that grow with $1/\sqrt{\delta}$; both requirements are far above the capability of any device available today.

The combination of these two ceilings is what closes the door on end-to-end execution. Three quantitative constraints explain why this problem is structural, not accidental:

- the qubit count grows as $\Theta(n \log n)$, so the smallest instance tabulated in the paper already crosses the practical ceiling of a state-vector simulator;
- the spectral gap δ is greater than or equal to 1 for all instances small enough to simulate, which makes $1/\sqrt{\delta}$ at most 1 and removes any speed-up window;
- the QPE precision ℓ_s derived from the same formula becomes too small to resolve the eigenphase of $|E\rangle$ from the other eigenphases of U_W.

Based on the present literature survey, no published work covers the combination "MNRS + Johnson graph + small instance + classical simulation + amplitude check". The flat distribution on $J(3,1)$ is therefore consistent with the literature, not anomalous.

5 Code-and-test mapping analysis

5.1 Method and reference framework

In parallel with the modifications, a structured comparison between the paper and the reference implementation was carried out. The comparison covers every algorithmic block described in the

paper — input preparation (Dicke state, vertex binary encoding), update operator U_U, marking oracle U_f, QPE-based approximate reflection, diffusion and output measurement — and locates, for each block, the corresponding sub-routines and the unit tests provided in the repository.

The evaluation is structured along the guidelines proposed by Moguel, Parejo, Ruiz-Cortés, Garcia-Alonso and Murillo [1] for empirical experiments in quantum software engineering. Their framework distinguishes two complementary axes: the *reporting guidelines*, which describe what a quantum experiment must declare in order to be reproducible (qubit and gate requirements, classical pre/post-processing, framework version, random seeds, error mitigation), and the *laboratory package* structure, which describes the artefacts that must accompany the experiment (scientific article, source code and dependencies, datasets, analysis scripts, log files, environment definition). We adopt the same nomenclature here and use the two checklists (Tables 4 and 5) as a methodological reference. The complete row-by-row evaluation that produced the two tables is preserved in the repository.

Two clarifications are in order. First, all "✓" rows in the tables below correspond to items that the original code or the paper already covers, so they are listed as a single line per area; only rows marked "∼" (partially met) and "✗" (not met) are reported individually, since those are the actionable signals. Second, the "Comment" column refers to the original `cssp.py` as published with the paper; the modifications described in Section 3 and the upstream pull request close some of these rows but not all, which is precisely the point of distinguishing addressed from open issues at the end of this section.

5.2 Reporting guidelines evaluation

Table 4 reports the evaluation along the reporting axis. The original paper and code already satisfy the context, the experimental design of the initialisation routines, and the threats-to-validity discussion, so those rows are collapsed into a single "✓ rows" summary line per area. The bootstrapping and platform-availability rows are not applicable to a deterministic local simulator and are omitted.

Area	Item	Status	Comment on the original code/paper
Context	Q-bits, gates, depth, limitations of platform	✓ rows	Polynomial qubit growth, depth analysis, NISQ feasibility limits all explicitly discussed in the paper.
	Connectivity between qubits	∼	The paper assumes an all-to-all topology; current hardware constraints are acknowledged but not modelled.
Experimental design	State-initialisation techniques (Dicke, W-states, VBE)	✓ rows	Cleanly implemented in `qregs_init.py` as described.
	Justification for the number of shots	✗	The script prints raw amplitudes; no shot count is set or motivated.
Execution	Quantum framework declared (`myQLM`)	✓ rows	The framework and the AQASM (Atos Quantum Assembly Language) language are declared.
	Pre/post-processing of classical data	∼	Sorting and bit-width computation are implicit in the script and are not formalised as separate steps.
	Pinned framework version / dependencies	✗	No `requirements.txt`; reproducibility on a different machine is not guaranteed.
	Random seeds for simulation	✗	No seed is logged; runs on different machines are not guaranteed to be bit-identical.
	Readout error mitigation	✗	Not applicable to a noiseless simulator, but should be declared explicitly.
Analysis	Statistical model of the output distribution	✗	Probabilities are printed but no variance/aggregation across runs is reported.
	Error correction or mitigation	✗	The paper assumes a fault-tolerant device; no QEC is modelled in the code.
Threats to validity	Effect of different hardware; extension to other platforms	✓ rows	Both points are addressed in the paper's discussion.

Table 4: Reporting-guideline evaluation of the original implementation, following Moguel et al. [1]. Rows marked "✓ rows" summarise items that the original code or the paper already satisfies; the others indicate gaps that the present analysis or the upstream pull request partially close.

5.3 Laboratory-package evaluation

Table 5 reports the evaluation along the laboratory-package axis. The original repository excels in the "Article" and "Experimental material" sections (the paper and the source code are both rigorously presented), but lacks several of the engineering artefacts that the framework requires for full reproducibility.

Section	Item	Status	Comment on the original repository
Article	Scientific article, design, context, authors	✓ rows	Peer-reviewed paper, complete authorship, conceptual diagrams.
Experimental material	Conceptual diagrams; full source code	✓ rows	The paper is rich in circuit diagrams and the source is openly hosted.
	Instructions for running the experiment	✗	No CI pipeline, no runner script and no formal "how to launch" guide.
Datasets	Pre-experimental data (built-in arrays, target)	✓ rows	Sample inputs are included in the script.
	Data processing (sorting, padding)	∼	Performed inline in the script, not exposed as a documented stage.
	Resulting datasets / log files	✗	No logs or output files are produced by the original repository; we created our own captures.
System	Hardware/QPU requirements; NISQ vs. FT (fault-tolerant) discussion	✓ rows	Section II of the paper covers the practical limits explicitly.
	Description of the classical software architecture	∼	Visible in the source but not formalised in the documentation.
Scripts	Inline description of the algorithmic decisions	✓ rows	Comments inside the routines explain the design choices.
	Analysis scripts (CSV, plots)	✗	No script processes the simulation output programmatically.
	Log file of the obtained results	✗	No structured logging is produced.
Dictionary	Public hosting of the artefacts	✓ rows	GitHub repository, public visibility.
	File/folder manifest	✗	No artefact dictionary; the role of every file must be inferred.
	Download/run instructions	✗	The original `README` did not list dependencies or commands; this gap is closed by the rewritten file.
	Explicit license file	∼	Implied by university policy but no `LICENSE` file is committed.
	Virtual environment / Docker	✗	No `Dockerfile` or pinned environment.
	Functionality verification (tests)	∼	Sub-routine tests exist; no end-to-end verification of `cssp.py`; no test for the *low-width* variant of the sliding-sort.
	DOI for the laboratory package	✗	The repository does not have an attached DOI.

Table 5: Laboratory-package evaluation of the original repository, following Moguel et al. [1]. The "✓ rows" summary lines collapse the items that the repository already provides; only "∼" and "✗" rows are reported in detail.

5.4 Critical points that have been addressed

The two evaluations above identify a set of issues that were agreed with the paper authors and closed via the upstream pull request that consolidates the project's correctness fixes.

Asymmetric reflections inside the QPE block. Both Reflection A and Reflection B now control the phase flip on the full coin register, with the corresponding X-sandwich applied symmetrically to both halves. This corresponds to Modification 1 of Section 3.

Degenerate QPE register on small instances. The lower bound on ℓ_s has been raised so that the QPE register is never trivial. The change is invisible on instances in which the original formula already returned a value above the new floor, hence fully backward-compatible. This corresponds to Modification 4.

Documentation gap between paper and code. The original `README` did not cite the paper, did not list the dependencies and did not describe the command-line interface of `cssp.py`. The rewritten file adds the citation, an explicit list of requirements (`myQLM`, `qat-utils`) and a usage block matching the actual flags accepted by the script.

5.5 Critical points that remain open

The other "∼" and "✗" rows of Tables 4 and 5 are not fixed by the present pull request and remain natural next steps.

End-to-end validation is structurally infeasible on a state-vector simulator. As recalled in Section 2, even the smallest instance reported in the paper exceeds the practical ceiling of `PyLinalg` on commodity hardware. This is not a defect of the code but a property of the algorithmic regime targeted by the paper.

Surgical reset of the cleanup ancillas. The removal of the destructive cleanup block (Modification 2) is a local mitigation: a per-element reset of α and ω that does not touch the entangled state would be the proper fix, but it requires a different construction and was therefore not included in the upstream pull request.

Test coverage of the composite walk operator. The existing tests cover the building blocks (`insert`, `delete`, oracle, Dicke preparation) in isolation, but there is no test that exercises the full walk operator U_W as a single unit, and there is no test for the *low-width* variant of the sliding-sort routine. The bugs identified in Modifications 1 and 3 would have been caught earlier by such tests.

Reproducibility artefacts. The repository does not provide a pinned environment (`requirements.txt`, `Dockerfile`), random seeds for the simulator, an explicit `LICENSE` file, an artefact dictionary, or a script for processing the output of `cssp.py` (CSV, plots). All of these are required by the laboratory-package framework of Moguel et al. and would significantly shorten the on-boarding time of any future contributor.

6 Conclusions

The project started as a reproduction attempt of the paper's algorithm on the smallest instance and turned, after a persistent flat output, into a structured diagnosis. Two genuine bugs were identified and confirmed by the paper authors: asymmetric QPE reflections (Modification 1) and a degenerate QPE register when $\delta > 1$ (Modification 4). The remaining three modifications were either local mitigations or diagnostic instrumentation. All five modifications together leave the output unchanged because $J(3, 1)$ sits outside the regime in which MNRS provides any speed-up.

The bugs confirmed as real by the paper authors were merged upstream as pull request #1 (`paper-codes/2025-TC`), together with a rewritten `README`. The flat output is not a defect of the code: it is a consequence of the gap between the algorithmic regime targeted by the paper and the largest instance a state-vector simulator can reach.

Future work could follow one of two paths: a direct Grover search on a compact vertex register, which validates the idea of amplitude amplification at small scale; or a stripped-down MNRS variant with an exact reflection, which keeps the walk structure without strict resource requirements. Either route would close the simulation gap while remaining connected to the paper's ideas.

Repository. https://github.com/jacopobellosi/Solving-the-Subset-Sum-Problem-via-Quantum-Walk-S

References

[1] E. Moguel, J. A. Parejo, A. Ruiz-Cortés, J. Garcia-Alonso and J. M. Murillo, *Quantum software experiments: A reporting and laboratory package structure guidelines*, arXiv:2405.04192, ACM, 2024.